Supplementary Material

BAPnP: A Barycentric Affine Invariant Linear Solver for Robust and Efficient Perspective-n-Point Pose Estimation

Peilin Luo, Yang Guo

Department of Mathematics, Northeastern University, Shenyang, China

This supplementary document accompanies the manuscript submitted to *The Visual Computer*.

Permanent link: https://github.com/lpl8848/BAPnP_Solver

1 Overview

This document collects supplementary experiments that were conducted in response to reviewer comments but are not included in the main manuscript. These experiments provide additional evidence supporting the robustness of BAPnP under adversarial scenarios that go beyond the standard evaluations in the manuscript.

All C++ source code required to reproduce these experiments is available in the public repository at https://github.com/lpl8848/BAPnP_Solver.

2 Localized Basis-Point Corruption with Distributed Reference

Context: Reviewer 1 raised the concern that BAPnP's reliance on four selected base points could constitute a structural vulnerability, and suggested oDLT (a fully-distributed weighted DLT method) as the appropriate reference point for evaluating this concern.

Setup: $N = 20$ points, Gaussian noise $\sigma = 2.0\,\text{px}$ applied to all points. An additional directional noise of increasing magnitude (0–20 px) is injected exclusively into the four points that BAPnP's greedy selector would choose as its affine basis. 2000 Monte Carlo trials per noise level.

Table 1: Localized basis-point corruption ($N = 20$, 2000 trials).

Algorithm	Baseline (0 px)		Extreme (20 px)		Type
	Rot. (°)	Trans. (%)	Rot. (°)	Trans. (%)	
BAPnP	0.365	0.054	2.293	0.343	4-pt basis
EPnP	0.476	0.065	2.366	0.332	PCA basis
oDLT	**0.278**	**0.066**	**1.105**	**0.287**	Fully distributed
BAPnP-GN (refined)	0.228	0.026	1.008	0.113	4-pt + GN
SQPnP (global opt.)	0.240	0.028	1.121	0.123	Global

Findings: (a) The reviewer's concern is empirically validated for the pure-linear stage: pure-linear BAPnP (2.29°) degrades more than fully-distributed oDLT (1.11°) under 20 px basis corruption. (b) The vulnerability is not unique to BAPnP: EPnP (2.37°) degrades nearly identically. (c) Gauss–Newton refinement resolves the issue: BAPnP-GN (1.01°) matches both oDLT (1.11°) and SQPnP (1.12°).

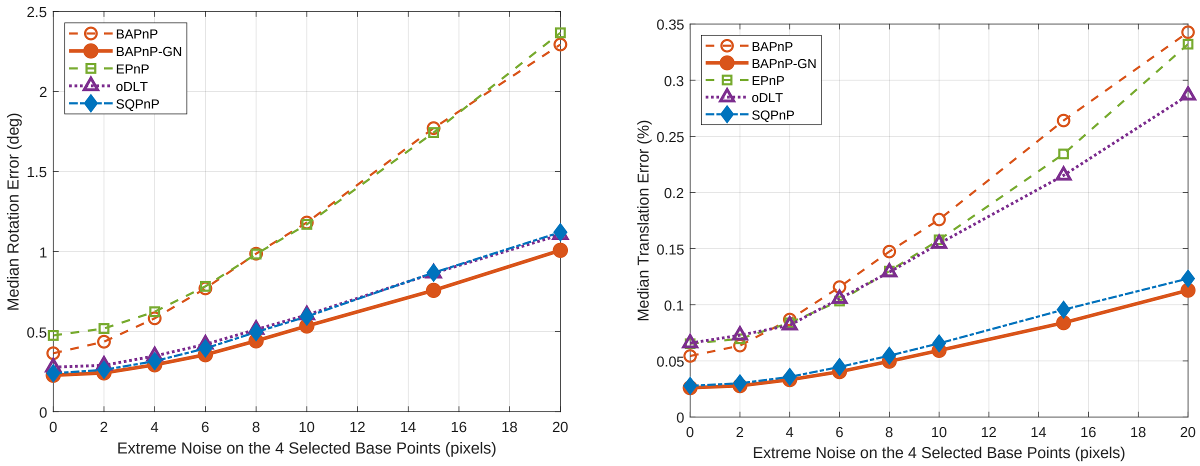

Figure 1: Localized basis-point corruption with oDLT as a fully-distributed reference. *Left:* Median rotation error. *Right:* Median translation error. 2000 Monte Carlo trials per noise level.

3 Minimal-Configuration Analysis ($N = 4, 5$)

Context: Reviewer 2 requested quantitative characterization of BAPnP's behavior at minimal point configurations ($N = 4, 5$), as the manuscript only states the $N \geq 6$ requirement without supporting data.

Setup: All 9 PnP methods from the manuscript evaluated at $N \in \{4, 5, 6\}$ across 7 noise levels ($\sigma \in [0, 3]$ px, 500 Monte Carlo trials each). Success: $E_{\mathrm{rot}} < 10°$ and $E_{\mathrm{trans}} < 0.5$ m.

Table 2: Minimal-configuration PnP comparison ($\sigma = 3$ px, 500 trials).

Method	$N = 4$		$N = 5$		$N = 6$	
	Rot(°)	Succ(%)	Rot(°)	Succ(%)	Rot(°)	Succ(%)
BAPnP	—	0	63.7	10	**1.43**	96
EPnP	23.6	67	8.0	84	2.47	93
SQPnP	3.5	94	1.3	98	**0.88**	**100**
CPnP	119.7	0	83.1	8	7.21	67
oDLT	128.4	0	119.7	2	9.28	91
OPnP	1.6	43	1.1	68	0.91	81
RPnP	2.2	91	1.4	96	1.19	98
SRPnP	**1.8**	**94**	**1.2**	**98**	0.99	99
MLPnP	103.9	15	82.4	31	18.9	85

Findings: At $N = 4$, BAPnP is structurally inapplicable (no non-base constraints). At $N = 5$, the single non-base constraint yields a rank-2 design matrix (nullity 2), causing the linear solver to extract an arbitrary vector from the nullspace (63.7° error). At $N = 6$, BAPnP achieves 1.43° and 96% success, confirming full functionality at the stated limit. For $N \leq 5$, dedicated minimal solvers (SQPnP, RPnP, SRPnP) should be preferred.

4 Extremal Outlier Injection Under RANSAC

Context: Reviewer 1 hypothesized that "clustered features on a facade with a few strong outliers on the extremal points" could break BAPnP's basis selection. This experiment directly tests that adversarial scenario.

Setup: Synthetic outliers are injected into the TUM RGB-D dataset: each outlier inherits the 3D coordinates of a randomly chosen real point but receives a deliberately wrong 2D projection at the image periphery (normalized coordinates 0.9–1.5 in absolute value). This simulates grossly mismatched SURF features on facade edges. Outlier ratios of 10%–50% (after-injection fraction) are tested. All four PnP solvers are evaluated within their own independent RANSAC loops (500 iterations, 6-pt minimal sampling, 2 px reprojection threshold, refit on best inlier set).

Table 3: Extremal outlier injection at 50% ratio (TUM RGB-D, 201 frames, 500 RANSAC iters).

Algorithm	Time (ms)	Rot. Err. (°)	Trans. Err. (m)	Succ. (%)	Inliers
BAPnP+RANSAC	**7.10**	1.469	0.035	**93.0**	390/988 (39.5%)
EPnP+RANSAC	25.07	1.453	0.035	92.5	385/988 (39.0%)
CPnP+RANSAC	17.83	1.445	0.034	84.1	288/952 (30.3%)
SQPnP+RANSAC	55.58	**1.476**	**0.035**	**93.5**	**392/986 (39.7%)**

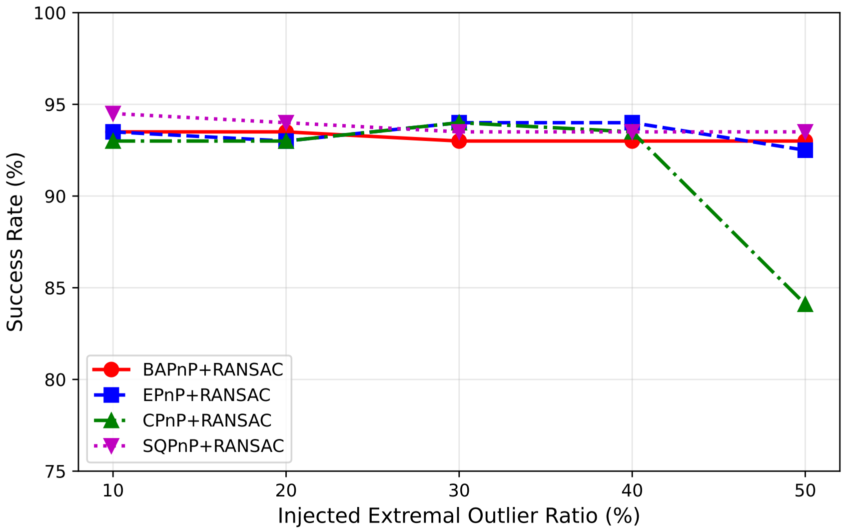

Figure 2: Success rate under increasing extremal outlier injection (10%–50%). BAPnP, EPnP, and SQPnP maintain 92–94% success across all ratios; CPnP degrades sharply at 50%.

Findings: (a) At 50% extremal outlier injection—far beyond any realistic feature-matching failure rate—BAPnP+RANSAC maintains 93.0% success, matching EPnP+RANSAC and SQPnP+RANSAC. (b) CPnP+RANSAC drops to 84.1%, confirming that its linear regressor is more vulnerable to extremal outliers. (c) BAPnP+RANSAC (7.10 ms) is 3.5× faster than EPnP+RANSAC (25.07 ms) and 7.8× faster than SQPnP+RANSAC (55.58 ms). The speed advantage is stable across all outlier ratios (10%–50%). (d) The full sweep shows consistent trends: BAPnP's robustness does not degrade disproportionately as outlier density increases, contrary to the hypothesis that the four-point basis creates a structural vulnerability.

5 Reproducibility

All experiments in this document can be reproduced using the C++ source files in the public repository:

- `main_tum_ransac.cpp` — Standard RANSAC-integrated TUM evaluation.

- `main_tum_high_ransac.cpp` — RANSAC evaluation with synthetic extremal outlier injection.

The MATLAB scripts for the synthetic experiments (localized basis corruption, minimal configuration analysis) are available in the `simulations/` directory of the repository.

Build instructions and dependency documentation are provided in `README.md`. The repository is archived with a permanent Zenodo DOI: https://doi.org/10.5281/zenodo.20080436.